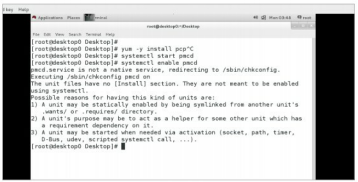

Figure 1: Package installation and starting the service

Once the connection is established with the Cockpit Web interface, a user must be authenticated in order to gain entry. Authentication is performed using the system's local OS account database. The dashboard screen in the Cockpit interface provides an overview of the core system's performance metrics. Metrics are reported on a per second basis, and allow the administrator to monitor the use of subsystems such as the CPU, memory, network and disk.

2. PCP

Red Hat Enterprise Linux 7 includes a program called Performance Co-Pilot, provided by the PCP RPM package. Performance Co-Pilot, or PCP, allows the administrator to collect and query data from various subsystems. It is installed with the *pcp* package. After installation, the machine will get the *pmcd* daemon, which is necessary for collecting the subsystem data. Additionally, the machine will also have various command line tools for querying system performance data.

There are several services that are part of PCP but the one that collects systems performance data locally is *pmcd*, or the Performance Metrics Collector Daemon. This service must be running in order to query performance data with the CP command line utilities. The *pcp* package provides a variety of command line utilities to gather and display data on the machine.

The *pmstat* command provides information similar to *vmstat*. *pmstat* supports options to adjust the interval between collections *(-t)* or the number of samples *(-s)*.

The *pmatop* command provides a top like output of machine statistics and data. It includes disk I/O and network I/O statistics, as well as CPU memory and process information provided by other tools. By default, *pmatop* will update every 5 seconds.

The *pmval* command is used to obtain historical statistics of per CPU idle time at one minute intervals from the most recent archive log.

3. Puppet

Puppet allows the systems administrator to write infrastructure as code using a descriptive language to configure machines, instead of using individualised and customised scripts to do so. Puppet's domain-specific

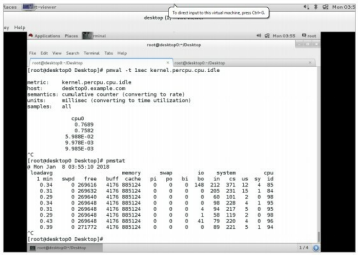

Figure 2: Output of *pmstat* and *pmval* commands

language is used to describe the state of a machine, and Puppet can enforce this state. This means that if the administrator mistakenly changes something on the machine, Puppet can enforce the state and return the machine to the desired state. Thus, not only can the Puppet code be used to configure a system initially, but it can also be used to keep the state of the system in line with the desired configuration.

Puppet architecture: Puppet uses a server/client model. The server is called a Puppet master and it stores recipes and manifests for the clients. The clients are called Puppet nodes and run the Puppet agent software. These nodes normally run a Puppet daemon that is used to connect to the Puppet master. The nodes will download the recipe assigned to the node from the Puppet master and apply the configuration if needed.

Configuring a Puppet client: Although Puppet can run in standalone mode, where all Puppet clients have Puppet modules locally that are applied to the system, most systems administrators find that this tool works best using a centralised Puppet master. The first step in deploying a Puppet client is to install the Puppet package.

```
# yum -y install puppet
```

Once the Puppet package is installed, the Puppet client must be configured with the host name of the Puppet master. The host name of the Puppet master should be placed in the */etc/puppet/puppet.conf* file under the *[agent]* section as shown below. First open the configuration file of Puppet, and then make entries as shown, before saving the file.

```
# vim /etc/puppet/puppet.conf
[agent]
Server=puppet.demo.example.com
```

The final step to be taken on the Puppet client is to start the Puppet agent service and configure it to run at boot time.

```
# systemctl start puppet.service.
```